

Rapport de Projet : Modernisation d'Infrastructure par l'Automatisation Cloud

Paul BELLANGER

1. Introduction et contexte

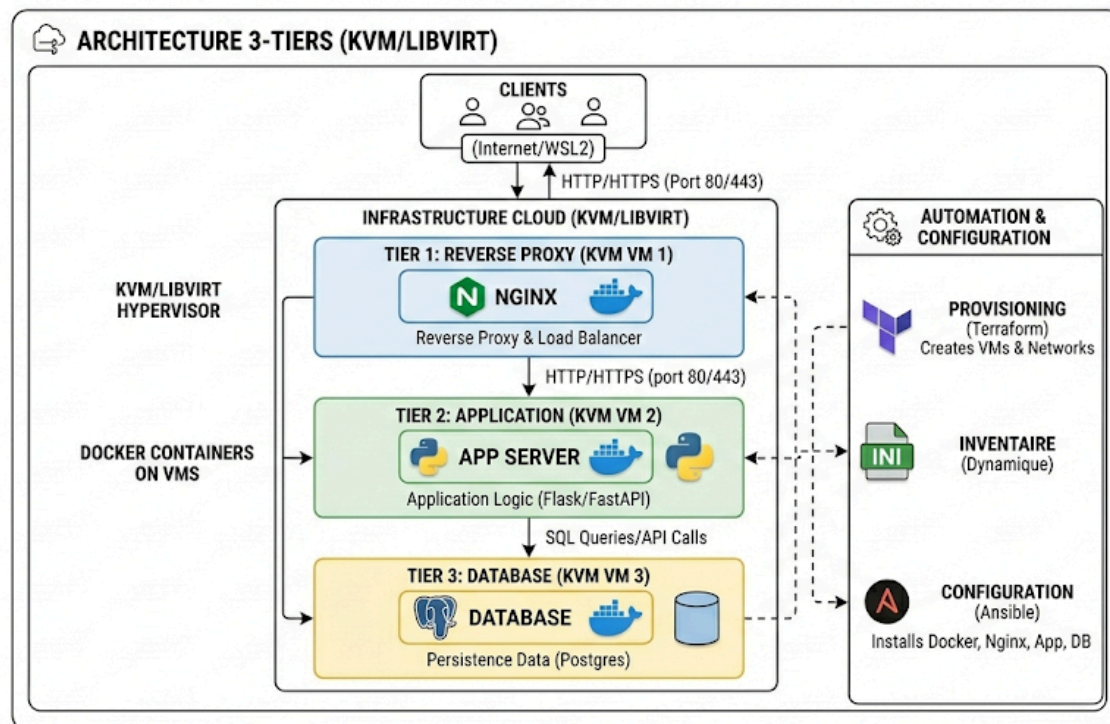
L'infrastructure informatique des entreprises est aujourd'hui confrontée à un besoin impératif d'agilité. Le modèle traditionnel de déploiement manuel sur serveur unique ("Snowflake servers") est devenu obsolète, car il limite la scalabilité, freine la maintenance et présente un risque critique de non-reproductibilité.

Ce projet vise à répondre à la problématique d'une PME cherchant à moderniser son application web interne. L'objectif est de transformer un environnement statique en une infrastructure **Cloud native**, industrialisée et hautement automatisée, en s'appuyant sur les paradigmes de l'Infrastructure as Code (IaC) et de la conteneurisation.

2. Présentation de l'architecture

L'architecture proposée repose sur le modèle **3-tiers**. Ce découpage permet une séparation stricte des préoccupations, garantissant que chaque composant est isolé, sécurisé et optimisé pour sa fonction spécifique.

- **Tier 1 (Reverse Proxy - Nginx)** : Cette couche sert de point d'entrée unique. Elle gère la terminaison SSL (potentielle), assure la sécurité périmétrique et effectue le routage vers l'application. Elle masque la complexité du réseau interne.
- **Tier 2 (Application - Flask)** : Ce serveur héberge la logique métier via le framework Flask. Le code est isolé du système d'exploitation par une conteneurisation Docker, garantissant l'indépendance de l'environnement.
- **Tier 3 (Base de données - PostgreSQL)** : La couche de stockage. L'isolation réseau empêche toute connexion directe depuis l'extérieur, seul le Tier 2 (Application) est autorisé à interroger la base.



3. Description de l'infrastructure virtualisée

Le choix de l'hyperviseur s'est porté sur **KVM/libvirt** pour sa nature open-source et sa parfaite intégration avec les outils d'automatisation.

- **Dimensionnement des ressources** : Chaque machine dispose de 512 Mo de RAM et de 10 Go de stockage (volumes indépendants). Ce dimensionnement est optimisé pour les besoins de l'application tout en restant économe en ressources système.
- **Gestion de l'état** : L'infrastructure est définie via **Terraform**. Chaque composant est typé et provisionné de manière déclarative. Le système de "clonage par disques indépendants" assure que chaque machine est unique et ne verrouille pas les autres.
- **Sécurisation au démarrage** : L'utilisation de **cloud-init** permet l'injection automatique de clés SSH éditables, assurant une gouvernance des accès dès la première seconde d'exécution.

4. Présentation du déploiement automatisé

L'automatisation est au cœur de ce projet pour garantir la reproductibilité.

- **Provisioning IaC (Terraform)** : Terraform gère le cycle de vie complet des machines : création, configuration réseau, attachement des volumes et démarrage.
- **Configuration Management (Ansible)** : Une fois les machines en ligne, Ansible orchestre la configuration logicielle :
 - Gestion des paquets système (**apt**).

- Installation et sécurisation du moteur Docker.
- Lancement des conteneurs via le module `docker_container`.
- **Bénéfice majeur** : Le "Time-to-Market" est réduit à quelques minutes, permettant de redéployer l'intégralité du datacenter virtuel en un simple clic.

5. Description de la conteneurisation

Le choix de **Docker** répond à trois impératifs :

1. **Encapsulation des dépendances** : Le conteneur Flask contient exactement la version de Python et les bibliothèques requises, éliminant les erreurs de type "ça fonctionne sur mon poste mais pas sur le serveur".
2. **Gestion du cycle de vie** : Les conteneurs sont configurés avec des `restart_policy: always`, garantissant la redémarrage automatique en cas de crash applicatif.
3. **Légèreté** : Contrairement à une VM, un conteneur partage le noyau de l'hôte, ce qui rend le déploiement quasi instantané.

6. Analyse et justification des choix techniques

Justification des choix

- **Architecture 3-tiers** : Choix délibéré pour la sécurité. En isolant la base de données, nous appliquons le principe du moindre privilège : si le proxy est compromis, l'attaquant n'a pas un accès direct à la base de données.
- **Terraform vs Manuel** : Terraform offre la gestion d'état (`.tfstate`), ce qui permet de savoir précisément quelles ressources sont en cours d'exécution.

Avantages vs Limites

- **Avantages** : Agilité, reproductibilité, standardisation des environnements (Dev, Test, Prod).
- **Limites** : L'architecture actuelle est une solution de référence ("proof of concept"). Pour une production réelle, il faudrait ajouter :
 - Une couche de **haute disponibilité** (Load Balancer devant plusieurs instances Flask).
 - Une stratégie de **sauvegarde automatisée** (backups de la base de données via `cron` ou volume dédié).
 - La gestion de **certificats HTTPS** pour garantir le chiffrement des flux.

7. Conclusion

Ce projet a permis de démontrer qu'une infrastructure Cloud, même modeste, bénéficie grandement des outils modernes d'automatisation. La solution mise en œuvre est cohérente, stable et entièrement reproductible. Elle constitue une base solide pour toute montée en

charge future. En respectant les principes de découpage des responsabilités et d'automatisation, nous avons transformé un processus manuel complexe en une chaîne de déploiement fluide et vérifiable, répondant ainsi parfaitement aux besoins de flexibilité de la PME.